RAILSMART · UK TRAIN RIDES

The Analysis Notebook, Explained

What was built in `data_analysis_and_cleaning.ipynb` — what each step does, how it was done, and why

Source data: National Rail ticket sales, Jan–Apr 2024
Scale: ~31,653 journeys · one row per ticket
Output: a clean, feature-rich table that feeds the dashboards, role views, forecasts and prototype models
Prepared: July 2026

How to read this document

The notebook moves in one straight line: it takes the raw ticket export, makes it trustworthy, adds the fields the project needs, then asks business questions of it and tests what can be predicted or forecast. This guide follows that same order. For each step you get three short notes:

WHAT the step produces. **HOW** it was done, in plain terms. **WHY** it was done that way rather than another.

Code is kept to a minimum — only the occasional column name or rule where it makes the explanation concrete.

31,653	42	18 → 24
CLEAN ROWS	COLUMNS AFTER ENGINEERING	ORIGINAL VS DERIVED FIELDS

Contents

1–2. Setup & loading the data

WHAT The standard toolkit is imported and the raw file `data/railway.csv` is read into a table (a pandas DataFrame).

HOW pandas/numpy handle the data, matplotlib/seaborn/plotly draw the charts, scikit-learn runs the later models. Display options are widened so long station names and pound prices render fully instead of being cut off.

WHY Everything downstream — every chart, model and export — reads from this one in-memory table, so it is loaded once and kept clean rather than re-read repeatedly.

3. Exploring the raw data

WHAT A first, no-assumptions look: the first rows, the table's shape, the column list, the data types, and summary statistics for both numeric and text columns.

HOW `head()` shows sample rows, `info()` reveals that every column arrives as plain text and flags which columns carry blanks, and `describe()` checks the numeric ranges (are any prices negative? are there stray categories?).

WHY You cannot clean what you have not looked at. This step establishes exactly which problems exist — blanks in three columns, everything typed as text — so the cleaning that follows is targeted rather than guesswork.

4–7. Cleaning the data

This is the heart of making the export trustworthy. Four distinct problems, each handled on its own logic.

4. Missing values — filled by meaning, not blanket-dropped

WHAT Three columns arrive with blanks; each blank is given the value it actually represents rather than being deleted.

HOW

- **Reason for Delay** — blank whenever the train ran on time. Labelled `On Time` so ~27k rows stay usable in group-bys. The reason labels also arrived inconsistently spelled (e.g. *Signal failure* vs *Signal Failure*) and were consolidated so one cause is not counted as several.
- **Railcard** — blank means the passenger held none. Filled with the literal `No Railcard` (deliberately not the word "None", which pandas would read back as a blank and silently undo the fix).
- **Actual Arrival Time** — blank only for the 1,880 cancelled trains. Left as missing on purpose: the train never arrived, so inventing a time would be dishonest. It is dealt with later in the duration logic instead.

WHY A blanket "drop all rows with blanks" would have thrown away tens of thousands of perfectly good journeys. Every blank here carries information, so preserving its meaning keeps the dataset both complete and honest.

5. Duplicates

WHAT A check for fully repeated rows and, separately, for repeated Transaction IDs.

HOW Transaction ID should be unique — one row per sale — so it is checked independently of full-row duplication.

WHY Duplicated sales would double-count revenue and journeys. The result was clean: no duplicates, so no rows were removed.

6. Price outliers

WHAT Unusually high fares are flagged — and then kept.

HOW The IQR (inter-quartile range) rule identifies statistical outliers; box-plot and histogram views confirm what they are.

WHY The expensive fares are genuine First Class / long-distance tickets, not data errors. Deleting them would understate real revenue, so they are documented but retained.

7. Data types

WHAT Text columns are converted to their proper types.

HOW Date columns become real datetimes; low-variety text columns (ticket class, purchase type, etc.) become the memory-efficient `category` type.

WHY Proper types shrink the memory footprint and make date maths and grouped summaries behave correctly instead of treating dates as strings.

8. Feature engineering

WHAT The fields the analysis actually needs are derived from the raw columns. This is where the dataset grows from 18 original columns to 42.

HOW New columns are computed from existing ones:

Derived field	Built from
Days-in-advance booking	Journey date − purchase date
Scheduled & actual journey duration	Arrival − departure times (with next-day arrivals handled)
Delay in minutes	Actual arrival − scheduled arrival
Is_Delayed (0/1 flag)	Journey Status
Refund_Requested (0/1 flag)	The text "Yes"/"No" in Refund Request
Calendar fields (month, weekday…)	Journey date
Time-of-day bands	Departure time (morning / afternoon / evening…)
Price-per-minute & price bands	Price and duration

WHY The raw file records facts (a purchase date, an arrival time); the analysis needs measures (how late, how far ahead, which part of the day). Turning "Yes"/"No" into a 0/1 flag, for example, is what makes claim rates and correlations computable later.

> **Worth knowing:** labels such as "Claimed", "Never claimed", "Refunds paid" and "Left unclaimed" that appear later in the notebook are **not stored columns**. They are row-subsets and price-sums computed live from `Journey Status`, `Refund_Requested` and `Price` at the moment the chart is drawn.

9. Exploratory analysis (EDA)

WHAT Seven themed views that describe what the cleaned data looks like, before any modelling.

HOW Each sub-section is a focused chart set:

- **9.1 Journey status** — the On Time / Delayed / Cancelled split (donut + bar).
- **9.2 Delay reasons** — for delayed trains only, what caused them.
- **9.3 Price & revenue** — average fare by ticket type, class and channel, and how price moves with booking lead time.
- **9.4 Timing & seasonality** — journey volume and delay rate by weekday and time-of-day band.
- **9.5 Payment & railcard** — payment-method mix and what a railcard does to the price paid.
- **9.6 Refunds** — how often refunds are requested and how that lines up with disruption.
- **9.7 Correlations** — a heatmap of what numeric fields move together.

WHY EDA turns a clean table into understanding. It surfaces the patterns (peak windows, delay causes, the railcard discount) that the strategic read-outs and models then quantify and act on.

10. Strategic read-outs

Business-level questions pulled straight from the cleaned data — the "so what" of the analysis.

- **10.1 Refunds actually requested** — the value of journeys where a refund *was* asked for. This is what the operator paid out.
- **10.2 Route reliability** — average delay per route, worst first. A minimum of 30 journeys per route is enforced so tiny routes (17 trips, 15 trips) don't top the ranking on noise. The same threshold is reused by the Station Manager dashboard and the Tableau workbook so all three agree.
- **10.3 The "urgency tax"** — what last-minute buyers pay versus those who book ahead (a 38.3% gap).
- **10.4 Unclaimed compensation** — the flagship finding. Of the 4,172 passengers *entitled* to compensation, only 26.8% claimed, leaving **£133,568** unclaimed in four months.

WHY 10.1 measures what was paid; 10.4 measures what was owed but never asked for — and the gap between them is the entire business case for the Refund Assistant product.

> **The number that matters:** £133,568 unclaimed is 18% of total revenue and **3.45× larger** than what the operator actually refunded. Nearly three in four entitled passengers never claim.

11. Presentation visualisations

WHAT Six polished, dashboard-ready charts.

HOW Hourly demand heatmap (hour × weekday); top-15 revenue routes; purchase channel × payment mix; railcard vs non-railcard price distribution; monthly revenue trend with growth rate; and a consolidated KPI summary screen.

WHY These feed the dashboards and slide deck directly. One cleaning fix lives here too: the railcard comparison had to test against the value `No Railcard`, not against "is the field blank" — because section 4 had already filled every blank, so the old "is it null" check labelled everyone a railcard holder.

12. Exporting the cleaned dataset

WHAT The cleaned, feature-engineered table is written to `cleaned_data/railway_cleaned.csv` alongside a short text summary.

HOW Saved with an explicit UTF-8 encoding (the Windows default would mangle the £ sign).

WHY This file is the single source of truth every downstream piece reads from — the dashboards, the three role-based views, and the models. Exporting once guarantees they all see identical numbers.

13. Prototype models

WHAT Four scikit-learn baselines showing what the cleaned data can support.

HOW Each model is scored honestly against a "do-nothing" majority-class baseline — the accuracy you get by always guessing the common outcome.

Model	Question	Result
13.1 Fare estimate	Predict the fare from route, class, ticket type, lead time	Strong — MAE £2.75, R^2 0.96; driven by route, not lead time
13.2 Delay risk	Will this journey run late?	Learnable — ROC AUC 0.98, recall 0.66
13.3 Cancellation	Why merging delay + cancellation hurts	Not predictable (AUC 0.73) — cancellations are near-random
13.4 Claim propensity	Among entitled passengers, who will claim?	AUC 0.86, recall 0.74 — the engine behind the Refund Assistant

WHY On a target where only 7% of journeys are delayed, 93% accuracy means the model learned nothing — so recall and AUC are quoted, never accuracy alone. The claim model is deliberately trained only on the 4,172 disrupted journeys: trained on *all* journeys it just rediscovers that on-time trains are never refunded (a definition, not a pattern). **13.5** then chains the models on a single journey the way the app would — estimate fare, score delay risk, and if disrupted, score claim likelihood to decide whether to nudge the passenger.

14. Forecasting

WHAT The project's three forecasting questions: how many rides next month, how much daily revenue, and how demand splits by ticket class.

HOW The 121-day daily series (1 Jan–30 Apr 2024) is checked for trend and weekly cycle, then a SARIMA model is backtested — trained on the first 91 days, forecasting the final 30 — against two naive benchmarks: a flat mean and a "repeat last week's matching weekday" rule.

WHY A model that cannot beat a flat line has not earned its complexity. Here demand turns out to be stationary (no trend, a weekday cycle spanning only ~11% of the mean), so the honest answer is the historical mean with a 95% interval — and because the ticket-class mix is stable to within half a percentage point, class demand needs no separate model, just a share of the total.

15. Summary & headline findings

Theme	Finding
Data quality	No duplicate sales. Blanks handled per column. Final file: 31,653 rows × 42 columns.
Punctuality	86.8% on time, 7.2% delayed, 5.9% cancelled → 13.2% disrupted.
Revenue	£741,921 total; £23.44 mean fare, £11.00 median (a First Class tail pulls the mean up).
The key number	Only 26.8% of entitled passengers claimed; £133,568 unclaimed = 18% of revenue, 3.45× refunds paid.
Delay causes	Weather 32.9%, signal failure 23.3%, staffing 19.4%, technical 16.9%, traffic 7.5%.
Booking behaviour	89% of tickets bought within 2 days of travel; none more than 28 days ahead.
Models	Fare estimation strong; delay risk learnable; cancellation not; claim propensity AUC 0.86.
Forecasting	Demand stationary Jan–Apr; honest forecast is the mean + 95% interval; class mix stable.

This cleaned table feeds the dashboards and the three role-based views (Data Engineer / Passenger / Station Manager). Every figure in this document traces back to `data_analysis_and_cleaning.ipynb`.